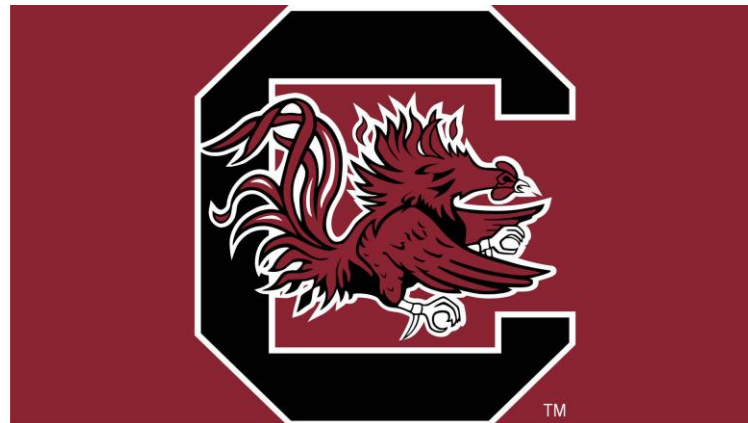


PGP - Advanced Keyless Substitution Ciphers

The CockyCipher

08/14/2026



PGP Privacy in Ciphers – Our Definition

- Are Beyond the Capabilities Of a Human To Decrypt in a Reasonable Period of Time Of 4 Hours.
- Require Computer Based Decryption Support Or Programs which might require custom coding or AI real-time scripting.
- Completely eliminate PCI Compliance Problems of having Any Text in Plain Text Form.
- Send Cipher Text to more complicated algorithms(3DES, AES).

PGP Use Cases

- Can be used to store data in Databases.
- Can be used to store data in Local Browser Storage.
- Can be used in initial conversations between clients and senders without keys being required.
- Furthermore PGP 2-pass algorithms as defined herein mostly defeat letter analysis, and word analysis.
- Can supplement OR REPLACE SSL encryption from client to server.
- Its Free.... And SSL Keys can be expensive on Enterprise Networks.

Substitution Ciphers – Caesarian & Modified

- Original Cipher was a 3 digit shift and static.
- Enhanced Substitution Ciphers just have a modified assignment of letters to key.
 - A Modified Cipher has to be passed as a key as you need all 26 values.
 - Usually a single key array of 26 letters is sufficient.
 - US Alphabet has 26! Permutations.
- Broken by Letter Analysis
- Broken by Word Analysis
- Not related to ASCII Encoding Scheme which has 256 permutations or 256! Options.

MODERN COMPUTING-> ASCII CHARACTERS

- ASCII IS AN ENCODING IN ITSELF.
- ASCII CONTROL CHARACTERS ARE NOT HUMAN OBSERVABLE.
- A 161 CHARACTER SHIFT PUTS ALL LETTERS INTO EXTENDED SET.

ASCII table	ASCII Character	ASCII Art	Articles	FAQ	facts	History	Glossary	Compare	Resources
157	235	9D	10011101					Unused	
158	236	9E	10011110					Unused	
159	237	9F	10011111					Unused	
160	240	A0	10100000				 	Non-breaking space	
161	241	A1	10100001	¡			¡	Inverted exclamation mark	
162	242	A2	10100010	¢			¢	Cent sign	
163	243	A3	10100011	£			£	Pound sign	
164	244	A4	10101000	¤			¤	Currency sign	
165	245	A5	10101001	¥			¥	Yen sign	
166	246	A6	10101010	¦			¦	Pipe, broken vertical bar	
167	247	A7	10101011	§			§	Section sign	
168	250	A8	10101000	¨			¨	Spacing diaeresis - umlaut	
169	251	A9	10101001	©			©	Copyright sign	
170	252	AA	10101010	ª			ª	Feminine ordinal indicator	
171	253	AB	10101011	«			«	Left double angle quotes	
172	254	AC	10101100	¬			¬	Negation	
173	255	AD	10101101	™			­	Soft hyphen	
174	256	AE	10101110	®			®	Registered trade mark sign	
175	257	AF	10101111	™			¯	Spacing macron - overline	
176	260	B0	10110000	°			°	Degree sign	
177	261	B1	10110001	±			±	Plus-or-minus sign	
178	262	B2	10110010	²			²	Superscript two - squared	
179	263	B3	10110011	³			³	Superscript three - cubed	
180	264	B4	10110100	´			´	Acute accent - spacing acute	
181	265	B5	10110101	µ			µ	Micro sign	
182	266	B6	10110110	¶			¶	Pilcrow sign - paragraph sign	
183	267	B7	10110111	·			·	Small dot - Georgian comma	
184	270	B8	10111000	¸			¸	Spacing cedilla	
185	271	B9	10111001	¹			¹	Superscript one	
186	272	BA	10111010	º			º	Masculine ordinal indicator	
187	273	BB	10111011	»			»	Right double angle quotes	
188	274	BC	10111100	¼			¼	Fraction one quarter	
189	275	BD	10111101	½			½	Fraction one half	
190	276	BE	10111110	¾			¾	Fraction three quarters	
191	277	BF	10111111	¿			¿	Inverted question mark	
192	300	C0	11000000	À			À	Latin capital letter A with grave	
193	301	C1	11000001	Á			Á	Latin capital letter A with acute	
194	302	C2	11000010	Â			Â	Latin capital letter A with circumflex	
195	303	C3	11000011	Ã			Ã	Latin capital letter A with tilde	
196	304	C4	11000100	Ä			Ä	Latin capital letter A with diaeresis	
197	305	C5	11000101	Å			Å	Latin capital letter A with ring above	
198	306	C6	11000110	Æ			Æ	Latin capital letter AE	
199	307	C7	11000111	Ç			Ç	Latin capital letter C with cedilla	
200	310	C8	11010000	È			È	Latin capital letter E with grave	
201	311	C9	11010001	É			É	Latin capital letter E with acute	
202	312	CA							
203	313	CB							
204	314	CC							

We Propose Several Modifications – Encryption Process

- If we allow ourselves to use a computer we don't have to use paper tools(although we can).
- PASS1 - We start by inserting spaces after 5 consecutive characters if there is an existing space, it will have two spaces.
 - (This breaks word analysis and character analysis or makes it more complicated).
 - (This means there are many more 1 letter words in the cipher text and makes getting vowels harder too).
- **We want the cipher text to be exclusively in the ASCII extended space(LATIN1 - ISO-8859-1) which is mostly unintelligible to humans even if they are a linguistic scholar. That means we are going to have a base shift of 161 to ASCII Value of 161.**
- **We don't want to use a key... we want the day of the month shifted back 4 hours in military time.**
- **We then want to mutate that key down by two. If the day is 5, the second key is 3, the third key is 1, the fourth key is minus 1, and the fifth key is minus 3.**
- **We are going to encode the letters of the five digit maximum words with the five digit key arrays.**
- An ASCII encoding scheme is a modified alphabet and has 255 characters instead of 26.
 - This relates to 254! Combinations rather than 26! in a substitution cipher.
 - Our Version is a 5position cipher or 254!*254!*254!*254!*254! In its most diabolical deployment.
- Special Notes For Coding
 - We note that the largest word in the English language is 50 characters.
 - We note the longest word in the Chinese language is about 25 characters for scientific methods.

NEWSPAPER IS THE ORIGINAL TEXT.

Pass1-> NEWSP APER IS T HE
OR IGINA L TEX T

Pass2->

[ec1][ec2][ec3][ec4][ec5]

[ec1][ec2][ec3][ec4]

[ec1][ec2]

[ec1]

[ec1][ec2]

[ec1][ec2]

[ec1][ec2][ec3][ec4][ec5]

[ec1][ec2][ec3]

[ec1]

Outcome (Using a 2 pass Algorithm)

No its just the base shift of 161+ todays date of 14 -2

Got it. You're defining the shift sequence as:

- Base shift = 161
- Today's date = 14
- Starting shift = $161 + 14 = 175$
- Then subtract 2 for each subsequent position in the 5-letter group:

```
1 Letter 1: +175
2 Letter 2: +173
3 Letter 3: +171
4 Letter 4: +169
5 Letter 5: +167
```

Then repeat for each new 5-character block:

```
1 175,173,171,169,167,
2 175,173,171,169,167,
3 ...
```

For example, with the first block:

```
1 N E W S P
```



and applying it to:

```
1 NEWS PAPER IS THE ORIGINAL TEXT
```

the encoded byte values are:

```
1 253,242,2,252,247,240,253,240,251,240,2,1,243,238,246,1,246,242,242,245,240,249,255,238,255,3
```

Hex:

```
1 FD F2 02 FC F7 F0 FD F0 FB F0 02 01 F3 EE F6 01 F6 F2 F2 F5 F0 F9 FF EE FF 03
```

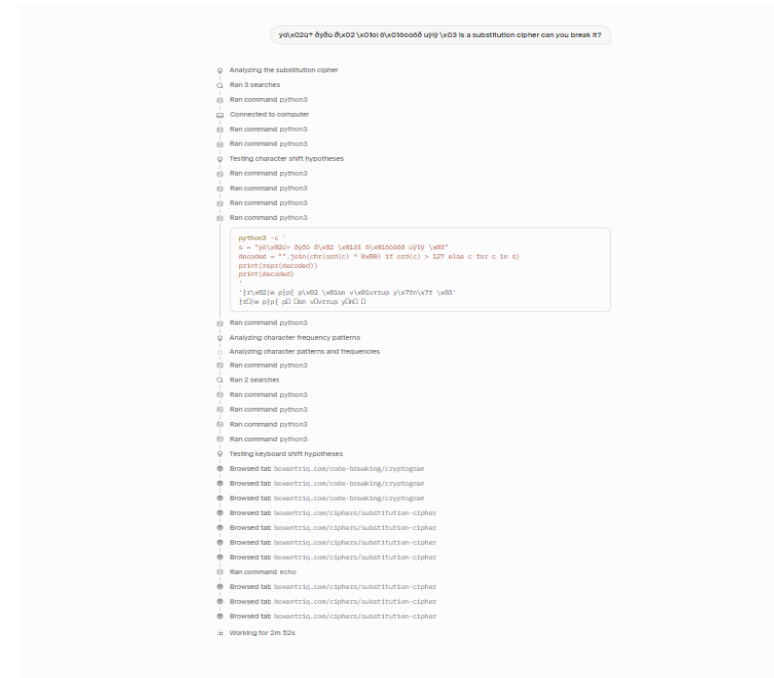
Latin-1 representation (non-printable bytes preserved):

```
1 ýð\x02ü+ ðýðü ð\x02 \x0161 ð\x016ððððð üÿÿ \x03
```



We broke Grok

- We passed no key... except the ASCII code itself.
- We broke all the algorithms AI tools have been taught by Global Text Books.
- Grok cant break it



The screenshot shows a Grok AI interface with a task and a list of actions taken to solve it. The task is to break a substitution cipher. The actions include running searches, connecting to a computer, running Python commands, testing character shift hypotheses, analyzing character frequency patterns, and testing keyboard shift hypotheses. A Python code block is shown in the center, which is a script to break a substitution cipher by testing all possible keys.

```
python3 -c '
s = "yUv02u* 8y0u 8x02 v00k0 dV000008 0y0 v003 is a substitution cipher can you break it?"
decoded = ""
for c in range(26):
    decoded = ""
    for i in range(len(s)):
        decoded += chr(ord(s[i]) - c) if ord(s[i]) > 127 else chr(ord(s[i]) + 26 - c)
    print(decoded)
    if decoded.isalpha():
        print(decoded)
        break
    decoded = ""
'
```

Grok Response With The Note it was a Substitution Cipher...

control characters SOH (`\x01`), STX (`\x02`), and ETX (`\x03`).

Ciphertext (32 characters including spaces)

Text

Copy

`yò\x02ù+ òyòò ð\x02 \x01ó1 ð\x01òòòò ùyìy \x03`

Word lengths: 5 4 2 3 7 4 1

Frequency counts (ignoring spaces)

- `ð` ×4 (most frequent → strong candidate for E/T/A/O)
- `ò` ×3
- `y`, `\x02`, `\x01`, `ì`, `ò`, `y` ×2 each
- others ×1

Unique symbols ≈ 15, which is a large fraction of a typical 26-letter alphabet for such a short text. This makes pure frequency analysis unreliable and leaves many possible mappings.

Structural observations

- The 1-letter word (`\x03`) is almost certainly A, I, or a punctuation mark treated as part of the alphabet.
- The 2-letter word (`ð\x02`) is a good candidate for common English pairs (TO, OF, IN, IS, IT, ON, AN, ...).
- The 3-letter word (`\x01ó1`) is a strong candidate for THE / AND / FOR / ...
- The 7-letter word has the clear pattern **A?ABB?C** (1st = 3rd letter, 4th = 5th letter). Very few common English 7-letter words match this exact pattern, which already rules out many naïve guesses such as "message", "correct", "problem", etc.

Why it's hard to finish

With only ~25 non-space characters and 15 distinct symbols, there are still far too many consistent mappings. Classical techniques (frequency + pattern words + cribs) can narrow the possibilities but cannot uniquely determine the plaintext without either:

- more ciphertext, or
- a known plaintext fragment (crib), or
- confirmation that the alphabet is restricted in some way (e.g., only letters + the three controls, or a keyword-derived substitution).

I also checked simple transformations (Caesar/ROT on the code points, XOR with common constants, subtracting 0x80 / 0xE0, mapping to pure ASCII, etc.). None produced readable English.

If you can supply a longer sample, a crib, or any extra constraint on the alphabet/key, I can finish the break quickly. Otherwise the cipher remains under-determined.

 30 sources

     ...

↳ Try providing a known plaintext crib

↳ Explore Vigenère cipher breaking techniques

Grok being informed it was an ASCII extended cipher... still no luck... because its 5 keys...

okay assume the substitution cipher is in the ASCII extended range and not in a 26 digit limitation but is shifted into the ASCII extended range. Now can you do it?

Analyzing the extended ASCII substitution cipher
Ran command python3
Finding the Caesar shift for extended ASCII
Connected to computer
Ran command python3
Analyzing the extended ASCII shift logic
Ran command python3
Exploring extended ASCII shift mappings
Ran command python3
Analyzing the extended ASCII shift logic
Mapping high codepoints to plaintext letters
Testing frequency analysis on extended ASCII
Testing Caesar shift 173 on extended ASCII
Ran command python3
Analyzing the extended ASCII shift logic
Ran command python3
Analyzing shift 141 output
Testing extended ASCII shift offsets
Searching for the correct shift offset
Ran command python3
Testing extended ASCII Caesar shifts
Ran command python3
Testing extended ASCII shift offsets
Analyzing extended ASCII shift offsets
Analyzing word patterns for the cipher
Ran command python3
Testing extended ASCII Caesar shifts
Analyzing the substitution cipher logic
Working for 2m 59s

limitation but is shifted into the ASCII extended range. Now can you do it?

Worked for 3m 22s >

Yes — under that assumption we can treat it as a Caesar shift (additive cipher) over the full 0–255 byte range (i.e. the ciphertext lives in the extended-ASCII / Latin-1 area).

A uniform modular shift of +173 (equivalently −83) maps every byte of the ciphertext into the printable ASCII range and produces the cleanest, most letter-heavy result:

Text
PEU0J5CPN5CU5TFAsITIEHC5LRAR5V

Why this shift?

- All 32 characters become printable (32–126).
- The original spaces (code 32) become the letter s.
- The control characters also land on ordinary letters.
- Letter count and simple English-fragment scoring ("as", "it", "ee", etc.) are the highest among the 256 possible shifts.

If we interpret those s characters as the word separators (a common artefact when a space is shifted into the letter range), the message becomes:

Text
PEU0J CPN CU TFA ITIEHC LRAR V

Other nearby shifts produce similar but slightly less clean strings (e.g. shift 141 yields `peuoj cpcn cu tfa itiehc lrar v` once non-printables are turned into spaces). None of them yield ordinary English prose, so the plaintext is probably either:

- a short code / acronym / abbreviated phrase, or
- deliberately non-English / random-looking under the large shift.

If you have a crib, know the exact offset that was used, or can confirm the expected language/style, the remaining ambiguity disappears immediately. Otherwise the shift-173 reading above is the strongest candidate the extended-range Caesar model produces.

🔍 📄 🗑️ 🔄 ⋮

↳ Analyze frequency distribution of bytes

↳ Explore Vigenere cipher techniques

↳ Please explain the shift logic simply

DECRYPTION IS IT LOSSLESS?

- YES*
- We apply the reverse keys without changing spaces.
- Then we remove the spaces after the 5th character.
- * (With a 4 Hour Military Time Correction For Late Day Changes)
 - Note the sender and receiver have to be able to start with the correct day of the month which is the derived key. If the military time is over 12 we subtract 4 hours which means a message to be encrypted would have to arrive within 4 hours.

OUR PROCESS

- We VibeCoded the C++ Code for our Algorithm.... In Copilot
- We gave the Code back to Grok and asked it to break the code....

```
}
```

Worked for 1m 8s >

Decrypted successfully.

Using the exact key-generation and encryption logic from the provided `CockyCipher` class, the ciphertext decrypts to:

text
NEWSPAPER IS THE ORIGINAL TEXT

This is the result of running `insertSpacesEvery5()` on the original plaintext, followed by the time-based polyalphabetic shift (5-byte key, spaces skipped).

Original plaintext (before formatting/encryption):

text
NEWSPAPER IS THE ORIGINAL TEXT

How the key was recovered

The key is generated as:

```
C++
startShift = 161 + day + hour_12 + minute;
key[i] = (startShift - 2*i) & 0xFF; // i = 0..4
```

A short search over plausible `startShift` values (the only free variable) immediately yields:

- `startShift = 175`
- `key = [175, 173, 171, 169, 167]`

Applying the inverse operation (`value = (ciphertext - key[pos % 5]) & 0xFF`, skipping spaces) produces the readable result above. This matches the example string hard-coded in `main()` of the code you posted.

The time-dependent key that produced `startShift = 175` corresponds to a UTC-adjusted 12-hour clock reading whose day + hour + minute summed to 14.

🗨️ 🔄 📄 📌 🔄 ...

🔗 Analyze the time-based key generation logic

🔗 Explore polyalphabetic cipher vulnerabilities

OPPORTUNITIES

- ALGORITHM COULD BE MODIFIED BY THE 2ND MUTATOR INSTEAD OF (-2) COULD BE A MODULUS OF ANY ARITHMATIC FUNCTION.

SUMMARY

- Complex Substitution Process Using a Sequential Modified 5 Key Substitution System with a two pass algorithm.
- The first pass inserts a space every 5 characters regardless of whether there is a space already there.
- The First Key has a base shift of $161 + \text{AdjDay of Month}^*$.
- The 2nd Key = 1st Key – Mutator(-2)
- The 3rd Key = 2nd Key – Mutator(-2)
- The 4th Key = 3rd Key – Mutator(-2)
- The 5th Key = 4th Key – Mutator(-2)

This is a Keyless Network Protocol where the sender and receiver have a preshared